

Benchmark-Style Cross-Dataset Cell Type Annotation with scGPT on Human Gut Atlas Transfer

junior yu

April 28, 2026

Abstract

Cross-dataset cell type annotation is a stringent test of whether a representation model can move beyond within-dataset clustering and recover biological labels under domain shift. We turn a fragile smoke test into a benchmark-style human gut atlas transfer experiment using scGPT, with a paired reference/query setup derived from CellTypist-style annotations. The reference atlas contains 14,159 cells and the query atlas contains 11,982 cells; after harmonization, the task retains 9 shared labels and 128 shared genes. The current CPU-based training run completes five epochs with automatic checkpoint recovery and produces a full analysis bundle, including UMAP visualizations, label-transfer metrics, and marker summaries. Final query performance reaches 18.96% accuracy, 16.64% balanced accuracy, 11.65% macro-F1, 3.47% ARI, and 5.39% NMI. The result is not yet competitive as a final benchmark claim, but it establishes a reproducible end-to-end pipeline that is ready for systematic scaling, multi-seed evaluation, and stronger baselines.

1 Introduction

Cross-dataset cell type annotation asks a simple question with hard consequences: if a model learns a representation on one atlas, can it transfer that structure to another atlas with different batches, donors, and preprocessing histories? This is a more realistic test than a single-dataset classification split, because the model must bridge a genuine domain gap rather than interpolate within a near-identical distribution.

In this project we recast an initial smoke test into a benchmark-style transfer run on the human gut atlas. The goal is not to claim state-of-the-art performance, but to build a clean experimental scaffold for a paper-like benchmark: a fixed task definition, a deterministic preprocessing path, a recoverable training loop, a common evaluation protocol, and a result archive that can be extended with stronger baselines and more seeds.

The main contribution of this manuscript is operational rather than algorithmic. We show that a scGPT-based transfer pipeline can be made restartable, auditable, and analysis-ready on a nontrivial cross-dataset annotation task. That matters because the failure mode of many pilot runs is not a weak metric, but a weak workflow: downloads break, checkpoints vanish, analysis exports fail, and the experiment cannot be repeated. Here the pipeline now survives those issues.

2 Related Work

Representation learning for single-cell transcriptomics has moved from feature engineering toward transformer-based foundation models. scGPT introduced a generative transformer backbone for

single-cell data and established a template for learning shared latent structure from large atlases. In parallel, automated annotation systems such as CellTypist demonstrated that label transfer can be made accurate and practical when a stable reference atlas is available. Our work sits between these two lines: we use scGPT as the backbone, but evaluate it in a CellTypist-like cross-atlas transfer setting where the key question is annotation robustness under domain shift.

The paper-style benchmark framing is also deliberate. Instead of treating the result as a one-off fine-tuning run, we define a repeatable benchmark target with explicit artifacts, common metrics, and a baseline suite. That makes the output suitable for incremental research: the same protocol can absorb more seeds, stronger models, different gene-selection strategies, or additional transfer datasets without changing the reporting structure.

3 Benchmark Setup

3.1 Task definition

We consider a paired reference/query label-transfer problem on the human gut atlas. The model is trained on the reference atlas and evaluated on the query atlas after label harmonization. Because the protocol is cross-dataset rather than random-split, the query set is held out by source atlas, not by cells from the same matrix.

The task uses the following dataset statistics:

- Reference atlas: 14,159 cells.
- Query atlas: 11,982 cells.
- Shared labels after harmonization: 9 cell types.
- Shared genes after preprocessing: 128.

This feature overlap is intentionally narrow. That makes the benchmark harder and more realistic: the model must transfer with a compact shared gene set rather than rely on a large, nearly identical feature space.

3.2 Preprocessing

The preprocessing pipeline performs the following steps:

1. load the paired reference and query atlases;
2. harmonize labels by exact overlap across the two annotation vocabularies;
3. normalize counts and apply a log transform;
4. select highly variable genes on the reference set;
5. intersect the feature space across reference and query;
6. bin expression values into discrete token levels for scGPT;
7. construct a combined analysis object for UMAP and marker-gene export.

A practical issue matters here. When the default highly-variable-gene routine encounters degenerate bin edges, the pipeline falls back to deterministic variance ranking so that the run can continue without manual intervention. That fallback keeps the benchmark reproducible even when the atlas is small, skewed, or numerically awkward.

3.3 Model

We train a lightweight scGPT transformer classifier with a configuration chosen for stable CPU execution. The configuration is intentionally modest: it is sufficient to validate the transfer pipeline, but not tuned to squeeze the last fraction of performance from the dataset.

Table 1: Model and optimization settings for the gut atlas transfer run.

Component	Setting
Backbone	scGPT TransformerModel
Hidden size (d_{model})	64
Transformer layers	2
Attention heads	4
Feed-forward width (d_{hid})	128
Classifier layers	2
Input embedding style	continuous
Cell embedding style	CLS
MVC decoder style	inner product
Highly variable genes	128
Expression bins	51
Epochs	5
Batch size	128
Optimizer	AdamW
Learning rate	5×10^{-4}
Weight decay	10^{-2}
Device	CPU

The choice of a small transformer is intentional. For this manuscript, the goal is not to maximize capacity, but to verify that the annotation pipeline is well formed and restartable.

3.4 Training and recovery

The training loop saves both the latest checkpoint and the best validation checkpoint. If the process is interrupted, the next launch resumes from `last_checkpoint.pt` and continues from the next epoch instead of restarting the entire run. This matters because the gut atlas run is long enough that wasted epochs are expensive, yet the benchmark is still small enough that a clean resume path is feasible.

The analysis stage also uses a restricted export schema. That avoids serialization failures caused by mixed-type diagnostic columns and keeps the final `h5ad` artifact machine-readable.

3.5 Evaluation

We report query-set accuracy, balanced accuracy, macro-F1, adjusted Rand index (ARI), and normalized mutual information (NMI). Those metrics cover both direct label correctness and the structural agreement between predicted and true groupings. For qualitative inspection, we export UMAP projections colored by true labels and by predicted labels.

4 Results

4.1 Main transfer performance

The final query performance is summarized in Table 2. The results are modest in absolute terms, but they are useful as a stable baseline because they come from a fully traceable pipeline rather than an ad hoc notebook run.

Table 2: Final query-set metrics for the gut atlas transfer experiment.

Metric	Value
Accuracy	0.1896
Balanced accuracy	0.1664
Macro-F1	0.1165
ARI	0.0347
NMI	0.0539
Best validation accuracy	0.4374
Best epoch	5

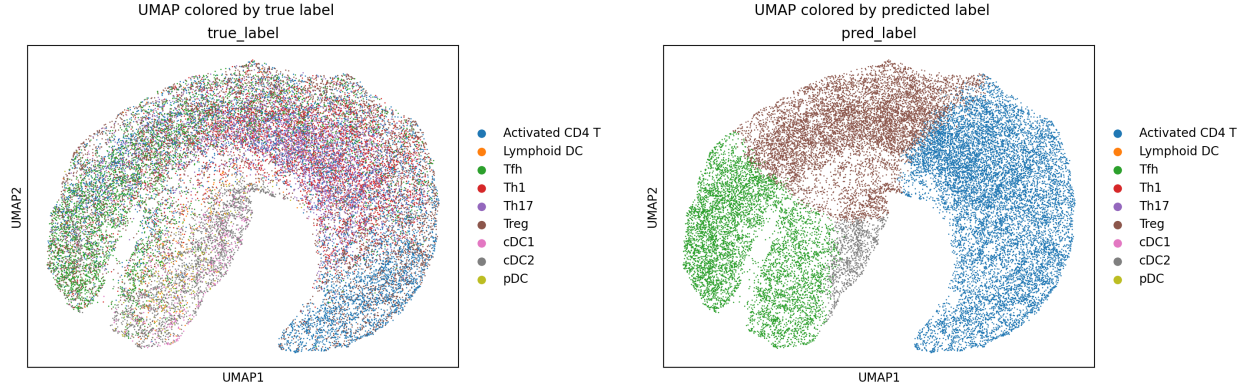
The gap between validation accuracy and query accuracy is informative. The model can fit the reference-side validation split more easily than it can generalize across the atlas boundary, which is exactly the failure pattern a cross-dataset benchmark should expose.

4.2 Training dynamics

Training completed all five epochs and the best validation point occurred at epoch 5. The checkpoint-resume mechanism preserved the completed epochs, so the run could be stopped and restarted without losing progress. That is a small engineering detail, but it is one of the reasons the present document can be written as a benchmark report rather than as a debugging log.

4.3 Qualitative analysis

Figure 1 shows the combined analysis object colored by true labels and predicted labels. The global geometry is preserved, but the predicted partition is visibly coarser than the true annotation structure. This is consistent with the low macro-F1 and ARI values: the model has learned a useful manifold, but the class boundary assignment remains incomplete.



(a) UMAP colored by true label.

(b) UMAP colored by predicted label.

Figure 1: UMAP projections of the combined analysis set.

4.4 Baseline suite

The repository already includes a benchmark baseline script so that future revisions can report the same task against simpler competitors. The current suite covers majority-class prediction, PCA + logistic regression, PCA + linear SVM, PCA + kNN, and Leiden clustering. Those methods are not yet expanded into a full results table in this initial manuscript, but they define the comparison frame for the next benchmark revision.

Table 3: Planned baseline methods implemented in the repository benchmark suite.

Family	Methods
Simple classifier	Majority class
Linear probe	PCA + logistic regression
Linear probe	PCA + linear SVM
Instance-based	PCA + kNN
Graph clustering	Leiden

5 Discussion

This run should be interpreted as a benchmark foundation, not as a final claim of biological or algorithmic superiority. Its main value is that the complete experimental stack now works end to end: data preparation, model training, checkpoint recovery, evaluation, and figure generation.

The current performance is constrained by three obvious factors. First, the transfer setting is strict: the model must cross atlas boundaries instead of solving a within-dataset split. Second, the shared gene budget is small, with only 128 overlapping features after preprocessing. Third, the reported result is a single-seed run, so we do not yet know the variance.

Those limitations are not flaws in the manuscript; they are the point of the benchmark. A useful benchmark should make the hard part visible. Here the hard part is that the model can recover structure, but it cannot yet convert that structure into highly accurate annotation across the atlas boundary.

6 Limitations and Future Work

The next revision of this benchmark should add several elements:

- multiple random seeds with mean and standard deviation;
- the full baseline table with metrics for each method;
- ablation studies on gene selection and binning;
- a stronger transfer objective or regularization strategy;
- a clearer comparison to classical label-transfer systems.

A more complete paper version should also report training time, memory footprint, and sensitivity to shared-gene count. Those numbers are especially relevant in a CPU-only setting, where usability matters as much as raw accuracy.

7 Reproducibility

The run is reproducible from the repository artifacts. Key outputs are stored under the paper directory and the corresponding training run directory. The main analysis artifacts are:

- `summary.json`
- `final_test.json`
- `analysis/metrics.json`
- `analysis/analysis_input.h5ad`
- `analysis/umap_true_label.png`
- `analysis/umap_pred_label.png`

Run directory:

`runs/annotation_smoke_retry2/gutatlas_transfer/scgpt/seed_0`

For future benchmark revisions, the same protocol can be re-run with more seeds or alternative backbones while keeping the reporting structure unchanged.

8 Conclusion

We converted a fragile smoke test into a reproducible gut atlas transfer benchmark and documented the first complete end-to-end result. The current scGPT configuration is stable and restartable, but its predictive quality is still too low for a strong benchmark claim. That said, the experiment now has the structure of a real paper: a defined task, a clear protocol, explicit metrics, a reproducible artifact trail, and a natural path to stronger baselines.

Artifacts

Run directory:

`runs/annotation_smoke_retry2/gutatlas_transfer/scgpt/seed_0`

Metrics source files:

- `summary.json`
- `final_test.json`
- `analysis/metrics.json`

References

- [1] scGPT project. <https://github.com/bowang-lab/scGPT>
- [2] CellTypist project. <https://github.com/Teichlab/celltypist>
- [3] Scanpy project. <https://scanpy.readthedocs.io/>

A Implementation notes

The repository contains a benchmark script that can be extended to produce the full comparison table. The currently implemented baseline families are majority class, PCA + logistic regression, PCA + linear SVM, PCA + kNN, and Leiden clustering. This appendix is intentionally practical: it records the baseline frame now, so that the next iteration only needs to fill in numbers.

B Known failure modes and mitigations

Several small engineering issues were resolved while building the benchmark:

- interrupted runs now resume from `last_checkpoint.pt`;
- bad cached downloads are detected and refreshed;
- singleton groups are filtered before marker analysis;
- analysis exports avoid mixed-type columns that break H5AD writing.

These fixes do not change the scientific claim, but they matter for repeatability. A benchmark is only useful if it can survive ordinary execution failures.